

ПЛАН АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ

American Eagle Python Automation

Объект: <https://www.ae.com/>

Параметр	Значение
Документ	Test Plan / Automation Test Plan
Проект	American_Eagle_python_automation
Ответственный	Илья Сидорычев
Версия документа	1.1
Дата актуализации	07.10.2026
Статус	Актуальный для текущей версии проекта

1. Цели автоматизации

- Ускорить проверку критически важных пользовательских и REST API сценариев.
- Снизить объём повторяющихся ручных проверок и влияние человеческого фактора.
- Ранее выявлять регрессионные дефекты после изменений в production-сайте и тестовой инфраструктуре.
- Обеспечить воспроизводимый запуск API- и UI-наборов локально и в CI.
- Формировать единый Allure Report с историей запусков и публиковать его через GitHub Pages.
- Поддерживать стабильный CI-набор и явно документировать сценарии, ограниченные anti-bot protection.
- Развивать самостоятельный Python AQA pet project как расширение профессионального портфолио вторым языком программирования.

2. Объект и контекст тестирования

American Eagle (ae.com) — реальный production e-commerce сайт. Python-проект охватывает Web UI и REST API, связанные с навигацией, поиском, каталогом, товарами, авторизацией и состоянием корзины.

- Тестирование выполняется против действующего production-окружения.
- Используются только безопасные сценарии, не приводящие к реальной покупке или оплате.
- Поведение сайта и доступность товаров могут изменяться независимо от тестового проекта.
- Anti-bot protection ограничивает отдельные автоматизированные сценарии регистрации и авторизации.
- Проект создан самостоятельно после обучения тестированию ПО и Python в Университете ИТМО; он не является обязательной частью учебного курса.

3. Scope — что входит в автоматизацию

3.1 UI

Область	Что проверяется	Категории
Home / Header / Footer	Открытие главной страницы, header, footer, основные элементы навигации, Shopping Bag, Favorites и Account panel.	UI, home_page, smoke/positive
Men's Clothes	Переход в каталог, сортировка Price: Low to High, фильтрация \$25-\$50, открытие товара.	UI, positive
Product / Shopping Bag	Выбор размера, добавление товара, цена, размер, quantity, subtotal, free shipping threshold, max quantity, удаление, два разных товара.	UI, positive
Search	Открытие поиска, поиск существующего товара и известное изменившееся поведение для несуществующего запроса.	UI, defect / xfail
Registration	Клиентские проверки формы: email, обязательные поля, пароль, zip code, дата рождения/terms. Успешная отправка ограничена anti-bot.	UI, account, negative / skip
Sign In	Проверки invalid/empty email. Успешный вход и password validation документированы, но часть серверных сценариев ограничена anti-bot.	UI, account, positive/negative / skip

3.2 REST API

Область	Что проверяется	Категории
Token API	Получение guest token; позитивная проверка access_token; запросы без/с неверной авторизацией.	API, auth

Browse API	Получение товаров категории и динамического productId; несуществующая категория; запрос без авторизации.	API, browse
Inventory API	Получение доступного товара и SKU; invalid product; запрос без авторизации.	API, inventory
Bag API	Lifecycle корзины: add / verify / update / delete; несколько товаров; invalid SKU; no auth; delete nonexistent; zero quantity.	API, bag

4. Out of Scope — что не тестируется

- Оформление реального заказа и любые операции оплаты.
- Проверка внутренних баз данных и прямой доступ к backend-хранилищам.
- Нагрузочное и стресс-тестирование production-сайта.
- Security / penetration testing.
- Деструктивные сценарии, способные повлиять на реальных пользователей или production-данные.
- Стабильное серверное подтверждение успешной регистрации/авторизации, если сценарий блокируется anti-bot protection.

5. Типы и pytest markers

Marker	Назначение
ui	Web UI тесты через Selenium WebDriver
api	REST API тесты через requests
smoke	Ключевая проверка конфигурации проекта
positive	Позитивные сценарии
negative	Негативные сценарии
defect	Известное изменившееся/дефектное поведение
account	Account / Registration / Sign In
home_page	Главная страница и её компоненты
auth	Token API
browse	Browse API
inventory	Inventory API
bag	Bag API

6. Тестовые данные и управление состоянием

- Guest token получается программно и используется API-сценариями.
- ProductId и доступный SKU подбираются динамически через Browse и Inventory API, чтобы снизить зависимость от жёстко заданных товарных данных.
- Перед/после Bag API тестов состояние корзины очищается fixture-логикой, чтобы повысить независимость сценариев.
- В UI тестах выбирается первый доступный размер вместо жёстко заданного значения.
- Для цены учитываются sale price и regular/list price; актуальная цена имеет приоритет.
- Credentials не хранятся в репозитории; AE_GUEST_AUTH локально передаётся через environment variable, в CI — через GitHub Repository Secret.

7. Технологический стек

Технология	Назначение
Python 3.12	Основной язык
Pytest 9.1.1	Тестовый фреймворк
Selenium WebDriver 4.50.0	UI-автоматизация
requests 2.34.2	REST API тестирование
Allure Pytest 2.16.2	Интеграция с Allure Report
Decimal	Точная работа с ценами
Git / GitHub	Контроль версий
GitHub Actions	CI
GitHub Pages	Публикация Allure Report
Chrome / Selenium Manager	Локальный и headless CI запуск
PyCharm	Среда разработки

8. Архитектура автоматизации

- Page Object Model для UI.
- Component-классы для Header и Footer.
- Steps-слой для переиспользуемых пользовательских бизнес-сценариев.
- API clients / helpers для REST API.

- Pytest fixtures для WebDriver, API session, clients и подготовки тестового состояния.
- Constants / config / utils для общей конфигурации и данных.
- Allure feature / story / title / step для отчётности.
- Pytest markers для гибкого запуска отдельных наборов.

9. Окружение

Среда	Описание
Локально	Windows 10 x64; Python 3.12.1; актуальный Google Chrome; PyCharm; virtual environment .venv.
CI	GitHub-hosted Ubuntu runner; Python 3.12; Chrome headless; Selenium Manager; GitHub Actions.
SUT	Публичный production-сайт American Eagle и публично доступные API, используемые проектом.

10. CI/CD стратегия

```
api-tests -> api-allure-results \
    -> allure-report -> gh-pages -> GitHub Pages
ui-tests -> ui-allure-results /
```

- Workflow запускается при push в main, pull_request в main и вручную через workflow_dispatch.
- API и UI выполняются в отдельных jobs.
- API job получает AE_GUEST_AUTH из GitHub Secrets.
- UI job запускает Chrome в headless-режиме при CI=true.
- Каждый job сохраняет собственный allure-results как artifact.
- allure-report job скачивает и объединяет API/UI artifacts, формирует отчёт и публикует его в gh-pages.
- GitHub Pages публикует отчёт по постоянной ссылке.

11. Allure Reporting

- Единый отчёт объединяет результаты API и UI.
- В отчёте доступны suites, test cases, feature/story/title, шаги, ошибки, stack traces, attachments и environment information.
- Для упавших UI тестов pytest hook прикладывает screenshot и current URL.
- История предыдущих отчётов сохраняется в gh-pages (до 20 запусков).
- Ссылка: https://ismeneger.github.io/American_Eagle_python_automation/

12. Anti-bot, skip и defect-стратегия

- Production-сайт использует anti-bot protection, которая может блокировать автоматизированную регистрацию и авторизацию.
- Успешная регистрация и успешный sign-in сохранены как предусмотренные сценарии и помечены pytest.mark.skip с явным reason.
- Password validation сценарии (empty / short / long / invalid password) предусмотрены в тестовой модели; в активном автоматическом наборе оставлен representative skipped scenario, остальные документируются в Test Plan/README, чтобы не раздувать отчёт однотипными skip.
- Сценарий добавления товара в Shopping Bag авторизованным пользователем также сохранён как предусмотренный UI-сценарий и помечен pytest.mark.skip, поскольку для его выполнения требуется автоматизированный sign-in, блокируемый anti-bot protection.
- Known search behavior для несуществующего запроса сохраняется как xfail/defect, поскольку сайт возвращает товары/рекомендации вместо пустого результата.
- Если production-поведение меняется, сначала определяется: дефект теста, дефект продукта или новая ожидаемая функциональность.

13. Риски и меры снижения

Риск	Влияние	Мера
Изменение production UI/API	Падение локаторов или контрактов	Page Objects/API clients изолируют изменения; анализ по Allure и логам.
Anti-bot protection	Блокировка account-сценариев	Skip с явным reason; сценарии документируются в Test Plan/README.
Динамические товары и цены	SKU недоступен, sale меняется	Динамический productId/SKU; sale/regular price logic.
Динамический DOM	StaleElementReferenceException	Explicit waits, повторное получение элементов, ограниченные retries.
Popup overlays	Перехват кликов	Централизованное закрытие popup в BasePage и повтор клика.
Сетевые/production сбои	Ложные падения	Повторный анализ; разделение API/UI jobs; artifacts и screenshots.
Секреты	Риск утечки	GitHub Secrets; чувствительные значения отсутствуют в репозитории.

14. Критерии успешного завершения

- API test suite проходит без необъяснённых ошибок.

- UI test suite проходит без необъяснённых ошибок с учётом намеренных skip и xfail.
- Критический Bag lifecycle завершается успешно.
- API и UI формируют Allure results.
- Allure results объединяются в единый отчёт.
- Allure Report успешно публикуется через GitHub Pages.
- CI остаётся воспроизводимым и секреты не попадают в Git-историю.

15. Текущее состояние набора

Набор	Текущее состояние
API tests	16 (16 passed)
UI tests	33 (28 passed, 4 skipped, 1 xfailed)
Smoke test	1 (passed)
Полный локальный набор	50 тестовых сценариев; последний полный локальный прогон: 45 passed, 4 skipped, 1 xfailed, 0 failed
CI	API и UI запускаются отдельно; smoke test остаётся локальной проверкой конфигурации

Количество сценариев может изменяться по мере развития проекта. При изменении набора README и Test Plan должны обновляться синхронно.

16. Поддержка и актуализация плана

- Test Plan обновляется при существенном изменении архитектуры, CI/CD, API/UI покрытия или внешних ограничений сайта.
- Конкретная версия Chrome не фиксируется как тестовое ограничение; используется актуальная поддерживаемая версия.
- При добавлении/удалении test cases рекомендуется синхронно обновлять README и данный Test Plan.
- Намеренные skip/xfail должны иметь понятное техническое или продуктовое обоснование.

17. Ответственный

Илья Сидорычев

GitHub: <https://github.com/ISmeneger>

Repository: https://github.com/ISmeneger/American_Eagle_python_automation